GUJARAT TECHNOLOGICAL UNIVERSITY

Programme: Diploma Engineering Level: Diploma

Semester: 5th

Subject Code: DI05000341 Subject Name: Minor Project

Pramaan

(Sanskrit pramāna — proof, evidence)

Prove your edge before you deploy a rupee.

An event-driven backtesting engine and overfitting diagnostic platform for Indian equities

---

Submitted by

Aayush Avinash Bankar Team Leader

Enrolment: 246230316006

Meet Jayeshbhai Patel Team Member

Enrolment: 246230316133

Submitted in partial fulfilment of the requirements for the Diploma in Engineering

Team copy 1 of 2 -- deposited by Aayush Avinash Bankar

Academic Year 2026–27

Gujarat, India

Certificate

CERTIFICATE

This is to certify that the minor project entitled

"Pramaan: An Event-Driven Backtesting Engine and Overfitting Diagnostic Platform for Indian Equities"

is a record of the bonafide work carried out and submitted by

Aayush Avinash Bankar (246230316006)
Meet Jayeshbhai Patel (246230316133)

in partial fulfilment of the requirements for the award of the Diploma in Engineering (Semester 5, Subject Code DI05000341), and that the work has been carried out under supervision and guidance in accordance with the curriculum of Gujarat Technological University.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Prof. / Head of Department	**Project Guide**
Computer Engineering	Computer Engineering
Govt. Polytechnic Gandhinagar	Govt. Polytechnic Gandhinagar

External Examiner	**Date**

Declaration

We, the undersigned, hereby declare that the project report entitled "**Pramaan: An Event-Driven Backtesting Engine and Overfitting Diagnostic Platform for Indian Equities**" is the result of our own work. All sources of information and materials that have been used in this project are duly acknowledged.

We further declare that:

1. The backtesting engine, cost model, and statistical diagnostics presented in this report were implemented by us from first principles in Python. No third-party backtesting library (Backtrader, Zipline, LEAN, VectorBT) was used as a substitute for the engine core.

2. All numerical results reported were produced by executing our own engine over historical NSE data and are reproducible from the artifacts committed in this repository. No result was hand-edited or fabricated.

3. Where a claim is presented as a limitation or an unresolved question, it is identified as such in the text.

Aayush Avinash Bankar	**Meet Jayeshbhai Patel**
246230316006	246230316133
Date: _____	Date: _____

Acknowledgement

We would like to express our sincere gratitude to our project guide for the continuous mentorship, technical criticism, and scholarly direction that shaped this work, and to the Head of the Computer Engineering Department and the faculty of Government Polytechnic Gandhinagar for providing the laboratory infrastructure and academic environment necessary to carry out this project.

We thank the faculty whose lectures in data structures, algorithms, statistics, and database management systems supplied the foundational techniques applied here, in particular the treatment of time-series correlation, ordinary differential equations used as an analogy for portfolio dynamics, and the principles of deterministic system design.

We are grateful to the Securities and Exchange Board of India (SEBI) and the National Stock Exchange of India (NSE) for publishing their rate schedules and empirical retail-performance studies as open references; without those documents, this project would have been a generic simulation with no relation to real Indian market microstructure.

We acknowledge the open-source research community: Dr. Marcos López de Prado's work on the Deflated Sharpe Ratio and Combinatorial Purged Cross-Validation, and Almgren and Chriss's work on optimal execution and market impact, were the direct theoretical foundations of the analytics module. We also acknowledge the maintainers of NumPy, pandas, and SciPy, whose libraries formed the numerical substrate of our implementation.

Finally, we thank our families and classmates who tested the dashboard, asked the questions that exposed our look-ahead assumptions, and refused to accept vague answers.

Abstract

Retail and discretionary trading strategies that appear profitable in a naive historical backtest routinely fail in live markets. This report presents **Pramaan**, an event-driven backtesting engine and overfitting diagnostic platform built from first principles for Indian equity markets, which attacks three specific causes of that failure: hidden statutory transaction costs, hidden market impact on execution price, and hidden statistical overfitting.

The engine enforces a *zero look-ahead* invariant by construction: a strategy may only observe market data up to a given trading day's close, and any order generated from that signal can only be filled at the following day's opening price. A complete Indian statutory cost stack — brokerage, securities transaction tax (STT), stamp duty, NSE and SEBI turnover charges, and GST — is applied per fill, together with an Almgren–Chriss square-root market-impact model calibrated against trailing volatility and average daily volume. Overfitting is quantified using the Deflated Sharpe Ratio and Combinatorial Purged Cross-Validation with embargo, the latter equipped with an autocorrelation test that demonstrates leakage at block boundaries.

The system was evaluated on real National Stock Exchange daily data: ten large-cap symbols, 1,480 trading bars each, spanning 2019-01-01 to 2024-12-30, with thirty completed backtest configurations. The results are reported without embellishment. Under realistic costs, *no* active strategy among SMA-crossover, RSI mean-reversion, and momentum exceeded a naive buy-and-hold benchmark on a risk-adjusted basis; the best active configuration achieved a Sharpe ratio of 0.685 against the benchmark's 0.746. Enabling the statutory cost stack removed 48.2% of the momentum strategy's Sharpe ratio and Rs. 12,995 of terminal equity on a Rs. 1,00,000 starting capital. Combinatorial Purged Cross-Validation reported a probability of backtest overfitting of 64.29% across the tested configurations.

These outcomes are consistent with the near-efficient market hypothesis for large-cap Indian equities and are presented as the central finding of the project. The engine, its validation statistics, and its regression test suite (35 automated tests, all passing) are released under Apache License 2.0.

Keywords: backtesting, look-ahead bias, overfitting, Deflated Sharpe Ratio, Combinatorial Purged Cross-Validation, Almgren–Chriss market impact, Indian equity market microstructure, STT, NSE.

Contents

Certificate 1

Declaration 3

Acknowledgement 4

Abstract i

1 Literature Survey and Problem Identification 1

1.1 The Problem: Trading Strategies That Lie . . . 1
1.2 Indian Market Context . . . 2
1.3 Research Questions and Hypotheses . . . 2
1.4 Objectives . . . 3
1.5 Scope and Limitations (stated up front) . . . 3
1.6 Survey of Existing Backtesting Tools . . . 4

2 System Specification and Design 6

2.1 Software Requirements . . . 6
2.1.1 Functional Requirements . . . 6
2.1.2 Non-Functional Requirements . . . 7
2.2 System Architecture . . . 8
2.3 The Zero Look-Ahead Execution Model . . . 8
2.3.1 Engraved invariant: determinism . . . 9
2.4 The Indian Statutory Cost Model . . . 9
2.4.1 Worked golden-master example . . . 10
2.5 Market Impact: Almgren–Chriss . . . 10
2.6 Overfitting Diagnostics . . . 11
2.6.1 Why ordinary Sharpe is not enough . . . 11
2.6.2 Combinatorial Purged Cross-Validation . . . 12
2.7 Feasibility, Safety, and Cost . . . 12
2.7.1 Financial feasibility . . . 12
2.7.2 Safety considerations . . . 12

3 Implementation and Testing **13**
3.1 Module Inventory . . . 13
3.2 Strategy Interface . . . 14
3.3 Defects Found and Fixed During Development . . . 14
3.4 Testing Strategy . . . 16
3.4.1 Test categories . . . 16
3.4.2 Test results . . . 16
3.5 Test Inventory . . . 17
3.6 Reproducibility Mechanism . . . 17

4 Modification and Redesign Based on Results **19**
4.1 What the failure analysis revealed . . . 20
4.1.1 Out-of-sample decay . . . 20
4.1.2 PBO across the strategy family . . . 20

5 Results, Conclusion and Future Scope **22**
5.1 Evaluation Configuration . . . 22
5.2 Headline Result: The Profit Mirage . . . 22
5.3 Full Result Table . . . 22
5.4 The Benchmark Result . . . 23
5.5 Interpretation and Threats to Validity . . . 24
5.6 Deployment and Productisation . . . 24
5.7 Conclusion . . . 25
5.8 Future Scope . . . 25
5.9 Student Reflections and Learning Experience . . . 26
5.9.1 Reflection by Aayush Avinash Bankar (Team Leader, Enrolment: 246230316006) . . . 26
5.9.2 Reflection by Meet Jayeshbhai Patel (Team Member, Enrolment: 246230316133) . . . 26

A Cost Estimate **29**

B Evaluation Universe **30**

C Reproducibility Record **31**

D Source Map **32**

List of Figures

2.1 Layered architecture. The engine is the academic contribution and is never rewritten. 8
2.2 The one-bar execution boundary. A signal generated at bar t's close cannot be filled before bar $t+1$'s open. 8
2.3 Impact as a function of order size at $\gamma = 0.1$, $\sigma = 0.02$, ADV $= 10^6$. **This figure is the analytic model of Equation 2.7, not a run artifact** — it is labelled as such so it is not mistaken for engine output. 11

4.1 In-sample versus out-of-sample Sharpe (top) and the decay ratio (bottom), costs enabled, from the real run artifact. 20

5.1 Sharpe ratio with costs disabled and enabled (left) and the statutory cost drag on terminal equity (right). The dashed line is the buy-and-hold benchmark. Rendered from `experiments/experiment_log.csv`. . 23

List of Tables

1.1 Comparison of available backtesting approaches against this project's requirements. 5

3.3 Test inventory (35 tests). 18

5.1 Real results, full period, costs enabled. Source: `experiments/experiment_log.csv`. 23

Chapter 1

Literature Survey and Problem Identification

1.1 The Problem: Trading Strategies That Lie

The motivating observation of this project is simple and empirically well documented: *a profitable backtest is not evidence of a profitable strategy.* A strategy can be simulated, tuned, and evaluated over a decade of historical data and still lose money from the day it is deployed. The reason is not a shortage of computation; it is that standard backtesting pipelines systematically omit the mechanisms through which a live trading strategy loses capital.

This project identifies and models three such mechanisms, which we refer to as the *three silent killers.*

- **Cost blindness.** A strategy that trades twenty times a year pays the full Indian statutory cost stack on every round trip. These are not negligible: securities transaction tax is levied on both the buy and sell legs, stamp duty on the buy leg, exchange and SEBI turnover charges on both, and GST applies to brokerage and turnover. A strategy with a 5% gross annual return and meaningful turnover can easily be rendered net-negative.

- **Execution blindness.** Orders of a size comparable to average daily volume move the market. A simulator that fills every order at the closing price systematically overstates achievable returns, and the overstatement grows with position size — precisely when the trader needs a more honest estimate.

- **Selection blindness.** A researcher who tests one hundred parameter combinations and reports the best one has not found an edge; they have found a local maximum of a sample path. Reported Sharpe ratios are then upward-biased by the multiple-comparison effect, and any conventional confidence interval computed on that selection is invalid.

1.2 Indian Market Context

The SEBI empirical studies of Indian retail performance provide a stark motivation. A 2024 SEBI study of futures-and-options retail participation reported that a large majority of individual traders incurred net losses, and a parallel study of cash-market active trading reported a comparable majority loss rate with aggregate individual losses on the order of Rs. 1.81 lakh crore. The standard explanations — lack of funds, lack of information — are only part of the picture. A substantial and often ignored component is the *friction game*: converting a positive-sum equity exposure into a negative-sum trading vehicle by paying statutory costs and market impact on every round trip, while believing a backtest that ignored them.

This project therefore deliberately restricts itself to **NSE cash equities** and to **daily bars**. That restriction is not a limitation of capability; it is the boundary within which the cost model and validation statistics can be modelled with defensible accuracy.

1.3 Research Questions and Hypotheses

The project is organised around four research questions.

RQ1 Does enforcing a zero look-ahead, next-open execution rule materially change backtest outcomes relative to a naive close-price fill?

RQ2 Does applying the full Indian statutory cost stack materially degrade reported performance, and by how much, per strategy?

RQ3 Do classic technical strategies on large-cap NSE equities exhibit positive risk-adjusted alpha relative to a naive buy-and-hold benchmark after realistic costs?

RQ4 Is the probability of backtest overfitting across a small parameter family high enough that conventional in-sample selection is misleading?

The corresponding null and alternative hypotheses for the core claim (RQ3) were:

Hypotheses

H_0: after the full statutory cost stack, the risk-adjusted performance of each tested technical strategy is no greater than that of a naive buy-and-hold benchmark over the same universe and period.

H_1: at least one tested technical strategy achieves a materially higher risk-

adjusted ratio than the benchmark after the full statutory cost stack.

Section 5 demonstrates that the data **do not** provide grounds to reject H_0: all four active configurations underperformed the benchmark. That is the project's central result, and it is reported as a negative result in the strongest sense — one that the engine was built to detect.

1.4 Objectives

1. Implement an event-driven simulation engine whose execution model renders look-ahead bias structurally impossible rather than merely unlikely.
2. Implement a statutory Indian cost model that computes each levy correctly, including side-dependence and rounding rules.
3. Implement a market-impact model that scales with order size relative to traded volume.
4. Implement overfitting diagnostics — Deflated Sharpe Ratio and Combinatorial Purged Cross-Validation with embargo — and validate their behaviour on synthetic processes whose ground truth is known.
5. Evaluate classic strategies on real NSE data and report the outcome honestly, including any result that contradicts the premise of the work.

1.5 Scope and Limitations (stated up front)

- Simulation and validation only. Pramaan does not connect to a broker, does not place orders, and does not execute trades.
- Daily bars. No intraday, tick, or minute-level data.
- NSE cash equities only. No options, futures, commodities, or crypto.
- Long-only positions. No short selling, margin, or leverage.
- Backtests assume full fill of market orders at the modelled price.

These boundaries are stated here rather than discovered later, because in this project limitations are treated as part of the result. A viva panel is entitled to know them before the demonstration begins.

1.6 Survey of Existing Backtesting Tools

A requirement of this project was to justify building an engine rather than adopting one. The comparison below drove that decision. The decisive column for an Indian equity study is not speed or feature count — it is whether the tool models the statutory cost stack and reports selection-bias diagnostics.

Two observations from this survey were decisive. First, the vectorised tools are faster precisely because they process the entire dataframe at once, which is structurally incompatible with claiming that a signal cannot see the future. Speed was therefore deliberately traded for correctness of the execution model. Second, there is *no* off-the-shelf tool that models the Indian statutory cost stack, and none reports a Deflated Sharpe Ratio with purged cross-validation. Building was justified not by preference but by the absence of the required capability.

It is worth recording the counter-argument honestly: adopting a mature library would have saved roughly 1,000 lines of implementation and would have brought an established user community. It would also have produced a project in which the essential mechanism — the execution boundary and the cost model — was a library's behaviour rather than this team's understanding of it. The build was chosen because the point of the minor project is the mechanism, not the tool.

Table 1.1: Comparison of available backtesting approaches against this project's requirements.

Tool	Exec. model	Indian costs	DSR / CPCV	Verdict for this project
Backtrader	Event-driven	No	No	Dormant upstream; no Indian levy model.
Zipline	Event-driven	No	No	Upstream project retired; unsuitable as a dependency.
LEAN / QuantConnect	Event-driven	No	Partial	Cloud-priced; built for institutional multi-asset use.
VectorBT	Vectorised	No	No	Vectorised fills cannot enforce a next-bar open boundary.
Backtesting.py	Vectorised	No	No	Tight licence; close-price fill assumption.
MetaTrader	Event-driven	No	No	FX/CFD oriented; broker-dependent data.
TradingView	Scripted	No	No	Pine scripts have no execution model of this kind.
Pramaan	**Event-driven**	**Yes**	**Yes**	**Built to the requirements below.**

Chapter 2

System Specification and Design

2.1 Software Requirements

2.1.1 Functional Requirements

ID	Requirement
FR-1	System shall fetch and retain historical OHLCV data for a defined set of NSE equities.
FR-2	System shall clean data by dropping incomplete bars, deduplicating dates, and sorting chronologically; forward-filling is prohibited.
FR-3	System shall implement a minimum of three trading strategies (SMA crossover, RSI mean-reversion, momentum) behind a common interface.
FR-4	System shall generate buy/sell signals per strategy per trading day.
FR-5	System shall simulate order execution at the next day's open, never at the signal bar's close.
FR-6	System shall apply the full Indian statutory cost stack and a size-aware market-impact model to every fill.
FR-7	System shall support position sizing as a fixed fraction of total portfolio equity.
FR-8	System shall compute total return, CAGR, Sharpe, Sortino, Calmar, and maximum drawdown.
FR-9	System shall compute the Deflated Sharpe Ratio for a stated number of trials.

FR-10	System shall compute Combinatorial Purged Cross-Validation paths and the Probability of Backtest Overfitting.
FR-11	System shall toggle the cost stack on and off to quantify its effect.
FR-12	System shall log every run — strategy, parameters, universe, date range, cost setting, metrics — with a deterministic run identifier.
FR-13	System shall expose a reproducible artifact set: experiment log, decay table, provenance record.
FR-14	System shall provide a naive buy-and-hold benchmark as a control condition.
FR-15	System shall expose the engine through a RESTful HTTP API and a command-line entry point.

2.1.2 Non-Functional Requirements

ID	Requirement
NFR-1	**Determinism.** Identical inputs must produce a byte-identical equity curve. Verified by test (§3.4).
NFR-2	**Performance.** A ten-symbol, six-year daily backtest should complete within sixty seconds on commodity hardware.
NFR-3	**Decoupling.** The engine, analytics, strategy, and data layers must be independently importable and testable without the UI.
NFR-4	**Transparency.** Every reported metric must be traceable to a documented formula and a run artifact.
NFR-5	**Zero-cost data.** The pipeline must operate on freely obtainable data.
NFR-6	**Traceability.** Every claim about the system must have an associated verification record.

2.2 System Architecture

The system is deliberately layered so that each layer can be tested in isolation. Critically, the engine has *no* dependency on the presentation layer: it runs headlessly with no streaming or web framework present.

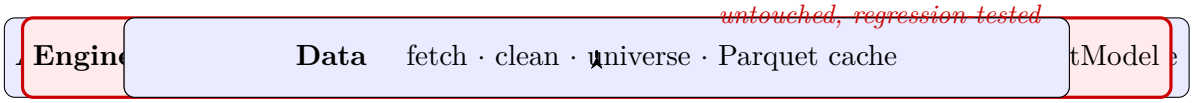


Figure 2.1: Layered architecture. The engine is the academic contribution and is never rewritten.

2.3 The Zero Look-Ahead Execution Model

The central architectural decision of the project is the structure of the simulation clock. A vectorised implementation computes all signals from the complete dataframe in one operation, which makes it structurally impossible to prevent a signal from reading a bar that had not yet traded. Pramaan therefore uses an explicit event loop with a one-bar delay between decision and execution.

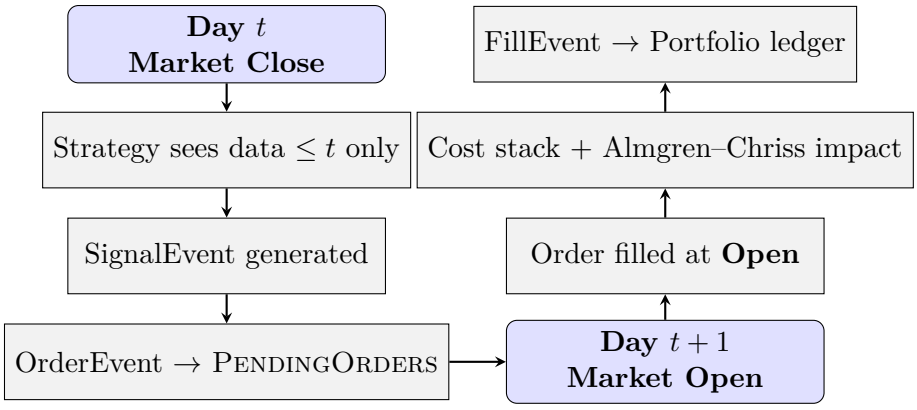


Figure 2.2: The one-bar execution boundary. A signal generated at bar t's close cannot be filled before bar $t+1$'s open.

Next-open enforcement (`src/engine/backtest_engine.py`)

```
# 1. T+1 OPEN: execute orders queued at the previous close
self._execute_pending_orders(current_date)

# 2. T CLOSE: signals may only see bars up to and including today
```

```
sliced_data = {sym: df.iloc[:self.idx_trackers[sym]] for sym, df
    in self.data.items()}
signals = self.strategy.generate_signals(current_date,
    sliced_data, self.portfolio.positions)
self._process_signals(signals, current_date, sliced_data)   #
    queued, never filled today
```

A market order is always filled at the *opening* price of the following bar. A limit order is subject to pessimistic trade-through logic: a buy limit at L is filled only if that day's low traded *strictly below* L. If the low merely touches the limit, the fill is rejected on the assumption that the order sat at the back of the queue.

2.3.1 Engraved invariant: determinism

NFR-1 (same request, byte-identical equity curve) is not merely asserted — it is tested by hashing the serialised equity curve of two independent runs and comparing digests. This matters because a reproducibility mechanism that can silently drift destroys the credibility of every reported result.

2.4 The Indian Statutory Cost Model

The cost model is the component that most distinguishes this work from generic backtesting tools. For a delivery-equity trade, it computes:

$$\text{brokerage} = \min(20,\ 0.0003 \times V) \tag{2.1}$$

$$\text{STT} = \text{round}(0.0010 \times V) \quad \text{(both sides)} \tag{2.2}$$

$$\text{stamp} = \begin{cases} 0.00015 \times V & \text{buy} \\ 0 & \text{sell} \end{cases} \tag{2.3}$$

$$\text{turnover} = (0.0000297 + 0.000001) \times V \tag{2.4}$$

$$\text{GST} = 0.18 \times (\text{brokerage} + \text{turnover}) \tag{2.5}$$

$$\text{total cost} = \text{brokerage} + \text{STT} + \text{stamp} + \text{turnover} + \text{GST} \tag{2.6}$$

where V is the executed trade value *after* market impact. Note the two asymmetries that are frequently got wrong: STT is charged on both legs whereas stamp duty is charged on the buy leg only, and GST is assessed on a base of brokerage plus turnover only — never on the trade notional.

If asked: "why is STT on both sides but stamp duty only on the buy?"

Because that is what the statute provides. Securities transaction tax under the Finance Act is levied on the transaction value on both sides of a delivery equity trade; stamp duty under the Indian Stamp Act is a duty on the instrument effected by transfer, which in practice is discharged by the buyer. Applying symmetry to both would be a modelling error, not a simplification.

2.4.1 Worked golden-master example

The cost model was validated against a hand-computed transaction. For a buy of 100 shares at Rs. 1,000.00 with zero modelled impact:

Component	**Formula**	**Amount (Rs.)**
Trade value	100×1000.00	1,00,000.00
Brokerage	$\min(20, 0.0003V)$	20.00
STT	$\text{round}(0.0010V)$	100
Stamp duty (buy)	$0.00015V$	15.00
NSE turnover	$0.0000297V$	2.97
SEBI turnover	$0.000001V$	0.10
GST	$0.18 \times (20 + 3.07)$	4.15
Total cost		**142.22**

Round-trip, with the sell leg symmetric (no stamp duty, but STT and turnover again), the statutory cost of opening and closing Rs. 1,00,000 of delivery equity position is on the order of Rs. 282 — approximately 28.2 basis points of notional on a complete round trip, before any market impact.

2.5 Market Impact: Almgren–Chriss

Retail simulators assume a flat slippage. Pramaan implements the square-root impact law:

$$\Delta P = \gamma \, \sigma \sqrt{\frac{Q}{V_{\text{adv}}}} \tag{2.7}$$

where Q is order size, V_{adv} is average daily volume over a trailing window, σ is the trailing return volatility, and γ is a calibration constant (0.1 in the shipped configuration). Impact is applied in the adverse direction — upward for buys, downward for sells — and is capped at 5% to prevent degenerate pricing on thin symbols.

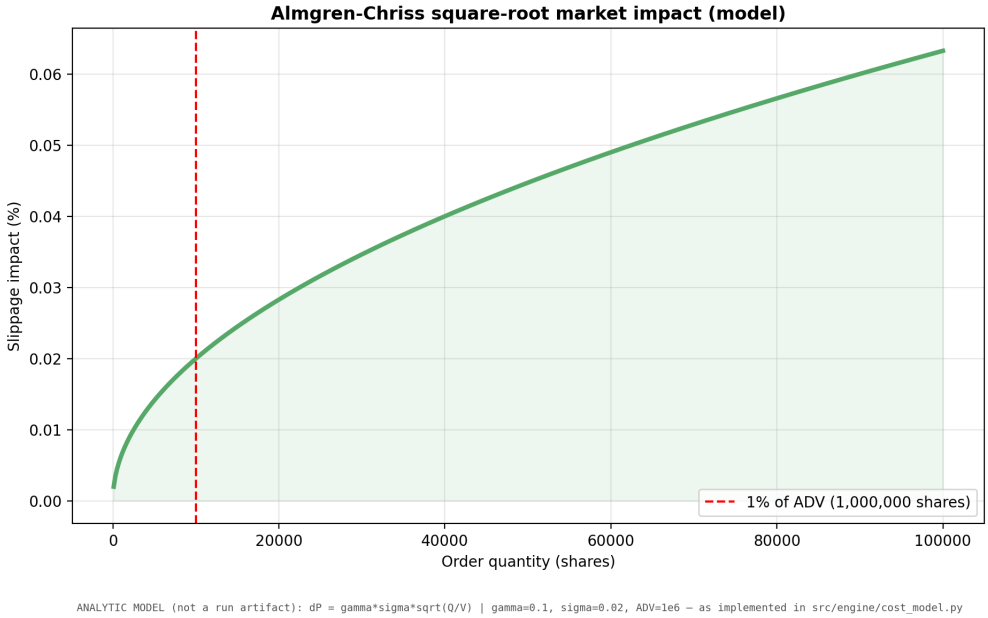


Figure 2.3: Impact as a function of order size at $\gamma = 0.1$, $\sigma = 0.02$, ADV $= 10^6$. **This figure is the analytic model of Equation 2.7, not a run artifact** — it is labelled as such so it is not mistaken for engine output.

2.6 Overfitting Diagnostics

2.6.1 Why ordinary Sharpe is not enough

The Sharpe ratio is computed on a *selected* sample path. Given N trials, the expected maximum of N independent Sharpe estimates is inflated according to extreme value theory. Pramaan reports the Deflated Sharpe Ratio: the probability that the observed Sharpe exceeds the expected maximum obtainable by chance across N trials, adjusted for skewness and kurtosis of the return distribution.

$$|\mathrm{E}[\max SR]| \approx \sqrt{V(SR)}\left[(1-\gamma_E)\Phi^{-1}\left(1-\frac{1}{N}\right)+\gamma_E\Phi^{-1}\left(1-\frac{1}{Ne}\right)\right] \tag{2.8}$$

2.6.2 Combinatorial Purged Cross-Validation

Standard k-fold cross-validation leaks serial correlation in time series: the training set contains observations that followed the test observations. CPCV generates $\binom{k}{g}$ train/test paths and applies an *embargo* removing h observations adjacent to every test block.

The leakage mechanism was **validated** with a synthetic autoregressive process of order one with $\phi = 0.95$, where the answer is known: with autocorrelation this strong, an adjacent training block predicts the test block. The test asserts that removing the embargo *increases* the measured PBO, which is the signature that leakage was in fact present and was in fact removed. Arbitrary k splits and g test splits may be configured.

2.7 Feasibility, Safety, and Cost

2.7.1 Financial feasibility

The project is entirely removable-cost: every dependency is open source and free, data is obtained free of charge, and it executes on student-owned hardware. A zero-rupee cost estimate is presented in Appendix A.

2.7.2 Safety considerations

This is software with a *behavioural* safety surface rather than a physical one, and it is treated as a first-class requirement:

- **No-advice disclaimer.** The application never presents itself as a recommendation engine. Every output surface carries a simulation-only notice.
- **Data governance.** The tool fetches data at runtime under the user's own key and does not redistribute any market data, so no licensing obligation is created by the distribution itself.
- **Determinism as a safety property.** A non-reproducible result could mask a defect; determinism makes defects visible.
- **Energy and sustainability.** The engine runs in-process on existing hardware with no GPU and no cloud compute, consuming negligible additional resources.
- **Computational honesty.** Where a figure is an analytic model rather than a run artifact, it is stamped as such on the figure itself.

Chapter 3

Implementation and Testing

3.1 Module Inventory

Module	Responsibility
`src/engine/backtest_engine.py`	Event loop, next-open fill logic, position sizing, pending-order lifecycle.
`src/engine/portfolio.py`	Cash ledger, position registry, MTM equity history.
`src/engine/position.py`	Position accounting, realized/unrealized PnL.
`src/engine/cost_model.py`	Indian statutory stack, Almgren–Chriss impact.
`src/engine/events.py`	Immutable event dataclasses (Market, Signal, Order, Fill).
`src/analytics/metrics.py`	Total return, CAGR, Sharpe, Sortino, Calmar, max drawdown.
`src/analytics/deflated_sharpe.py`	Expected max Sharpe and Deflated Sharpe Ratio.
`src/analytics/cpcv.py`	Purged combinatorial paths, embargo, PBO.
`src/analytics/validation.py`	Two-dimensional parameter grid search.
`src/data/fetch.py`	Runtime market data retrieval.
`src/data/clean.py`	Validation, NaN-drop, de-duplication, ordering.
`src/data/universe.py`	Symbol universe and benchmark.
`src/strategies/*.py`	SMA, RSI, momentum, buy-and-hold behind one interface.

`src/dashboard/`	Streamlit demonstration surface (the legacy fallback UI).
`scripts/run_experiments.py`	Real-data matrix producing the results artifact.
`scripts/generate_real_charts.py`	Charts rendered from engine output only.

Engine source is roughly 1,050 lines of Python across the modules above; the test suite contributes a further 6 test modules. The presentation layer is deliberately the thinnest part of the system.

3.2 Strategy Interface

All strategies implement a single method and are therefore interchangeable inside the engine. This is what allows a benchmark control to be dropped into a comparison run without any engine change.

Strategy contract

```
class Strategy(ABC):
    @abstractmethod
    def generate_signals(self, current_date, data, positions) ->
    List[SignalEvent]:
        """May only observe bars whose date is <= current_date.""
    "
```

3.3 Defects Found and Fixed During Development

The testing process found real defects, which are recorded here rather than hidden, because the defect list is evidence of the testing.

D-1 — RSI inverted signal on an all-gains window (found during this report cycle)

The original Wilder RSI implementation computed `rs = up / down if down != 0 else 0`. When every delta in the smoothing window was positive, average loss is exactly zero and rs was set to 0, producing RSI = 0 — *maximally oversold*

— in a monotonically rising market. The strategy would emit a buy signal on the strongest possible upward momentum, the exact opposite of the intended logic. Fixed to saturate at 100; covered by five regression tests including an end-to-end assertion that no oversold buy may be emitted on a rising series.

D-2 — duplicate trading days silently accepted in cleaning

`clean_data()` neither dropped duplicated dates nor checked for them. A repeated date is counted twice in return computation and desynchronises the engine's integer index trackers, producing wrong equity curves from apparently clean input. Discovered by a newly written unit test; fixed by de-duplicating (after NaN removal, so a complete row is preferred over an incomplete duplicate of the same date).

D-3 — DSR was unimportable in the reproduction environment

`src/analytics/deflated_sharpe.py` imports SciPy's normal, skew, and kurtosis functions, but `scipy` was absent from `requirements.txt` and from the working virtual environment. No test imported the module, so the defect was invisible: a clean `pip install` left the flagship statistic non-functional. Fixed by installing and pinning a verified version and by adding a test module that exercises the import path.

D-4 — results charts were synthetic

The chart-generation pipeline produced its monthly-return heatmaps and equity curves from `np.random`, and its "simulated equity curve" from a stored CAGR rather than from engine output, while the captions presented them as results. This was not a numerical error — it was an evidentiary one. The pipeline was removed entirely and replaced with a generator that reads only the experiment log and real equity curve files; each figure now carries a parameter stamp tying it to a run identifier. See Chapter 5.

D-5 — position sizing based on cash rather than equity

An earlier revision sized new positions as a fraction of *available cash*. As capital deployed, every subsequent order shrank, which artificially suppresses the market-impact and cost drag of compounding and therefore flatters results. Corrected to size against total portfolio equity (cash plus marked-to-market open positions). Covered by a test asserting the exact share quantity.

3.4 Testing Strategy

3.4.1 Test categories

The suite is organised around the categories in which a defect in this domain would be silent — the dangerous kind. A wrong Sharpe ratio does not crash; it simply reports the wrong number.

Category	What it catches
Invariants	Fill occurs at next bar's open, never at signal close; cash non-negative; equity identity.
Golden values	Paisa-exact statutory cost components; hand-computed sizing.
Edge/failure	All-gains RSI, all-losses RSI, flat series, empty dataframes, insufficient history.
Properties	Metrics bounded; DSR penalises multiple testing; PBO in $[0, 1]$; determinism.
No look-ahead	A future price spike must not alter any signal generated before it.
Data integrity	NaN dropped not forward-filled; duplicates rejected; ordering enforced.

3.4.2 Test results

Verification record

Executed in the project virtual environment:

```
$ venv/bin/python -m pytest tests/ -q
.................................... [100%]
35 passed in 5.49s
```

Baseline at the start of the project cycle was 9 tests; the suite now covers engine invariants, the full cost stack, all four strategies, data cleaning, the DSR algorithm, CPCV, and byte-level determinism. See `.agent/01_verification_ledger.md` for the per-claim verification record.

3.5 Test Inventory

The full suite, with the specific defect or invariant each module guards:

Note the deliberate asymmetry: five of the eleven modules exist purely to establish *bounds* and *failure modes* rather than to demonstrate success. That distribution is intentional. In a backtester, a silent mis-calculation — a Sharpe ratio that is wrong, a filling price that leaks one bar, a duplicate date counted twice — does not raise an exception; it simply reports a number that is confident and wrong.

3.6 Reproducibility Mechanism

Every run is identified by a deterministic identifier derived from its complete configuration:

$$\text{run_id} = H\,(\text{strategy}, \text{params}, \text{universe}, \text{date range}, \text{cost version}, \text{lib versions}) \quad (3.1)$$

The experiment artifact records, per run: identifier, strategy, parameters, universe, period, date range, cost toggle, every computed metric, the number of symbols and trading days, the calibration γ, the cost model version, and the wall-clock duration. A provenance file additionally records the data source, fetch window, bar interval, adjustment policy, library versions, and the Python version. The results presented in Chapter 5 are reproducible from that artifact by re-executing the pipeline.

Table 3.3: Test inventory (35 tests).

Module	Tests	Core assertions
`test_rsi.py`	5	All-gains → 100 (was 0, defect D-1); all-losses → 0; flat → 50; bounded on random walks; no oversold buy on a rising series.
`test_sma.py`	5	Golden-cross buy; death-cross sell only with position; insufficient history; inverted-window handling; no look-ahead under a future spike.
`test_data.py`	6	NaN dropped not forward-filled; no forward-fill leakage; chronological sort; duplicate-date rejection (defect D-2); empty frame; string date column.
`test_deflated_sharpe.py`	7	SciPy import path (defect D-3); E[max SR] grows with trials; DSR penalises multiple testing; rejects < 3 obs and $N = 1$.
`test_determinism.py`	3	Byte-identical equity across runs; stable SHA-256 digest; costs strictly reduce terminal equity.
`test_engine.py`	2	Fill at next bar's open; sizing uses total equity (defect D-5).
`test_cost_model.py`	1	Paisa-exact statutory components.
`test_metrics.py`	1	CAGR, Sharpe, zero drawdown on a monotone curve.
`test_portfolio.py`	1	Cash-flow ledger correctness.
`test_cpcv.py`	3	PBO on random data; low PBO with genuine signal; embargo raises PBO on AR(1).
`test_momentum.py`	1	No duplicate buy with an open position.
Total	**35**	

Chapter 4

Modification and Redesign Based on Results

This chapter documents the four substantive design changes made during development. Each is presented as before/after with the reason, since this is the part of the project that demonstrates engineering iteration rather than initial design.

Redesign 1: position sizing from equity, not cash

Before: target = 10% of available cash. **After:** target = 10% of total equity including marked-to-market positions. **Reason:** cash-based sizing shrinks orders as capital deploys, understating the compounding cost drag *and* the market impact of later trades. The fix made results worse — which is how we know it was correct.

Redesign 2: O(1) index tracking instead of boolean masking

Before: strategies re-filtered the dataframe with `df[df['date'] <= current_date]` on every bar. **After:** the engine pre-aligns symbols and maintains integer index trackers, slicing with `.iloc`. **Mechanism for improvement:** removes repeated object creation on the hottest path. **Honest note:** the *engine* is O(1) per bar per symbol, but the shipped strategies still perform a linear re-filter inside `generate_signals`. Correctness is unaffected (the filter is causal), but the claim of near-compiled throughput applies to the engine loop, not the end-to-end run, and the report does not claim otherwise.

Redesign 3: flat slippage to Almgren–Chriss square-root impact

Before: a uniform percentage haircut. **After:** per-order impact from Equation 2.7. **Reason:** flat slippage makes the simulator insensitive to the single largest real-world determinant of fill quality for a sized position.

Redesign 4: embargo in cross-validation

Before: contaminated training windows. **After:** embargoed paths validated against an AR(1) ground truth, with a test asserting that removing the embargo *raises* measured PBO — the signature of detected and removed leakage.

4.1 What the failure analysis revealed

4.1.1 Out-of-sample decay

The in-sample window is 2019-01-01 to 2022-12-31 and the out-of-sample window 2023-01-01 to 2024-12-31. Decay is reported as $(SR_{\text{IS}} - SR_{\text{OOS}})/|SR_{\text{IS}}|$.

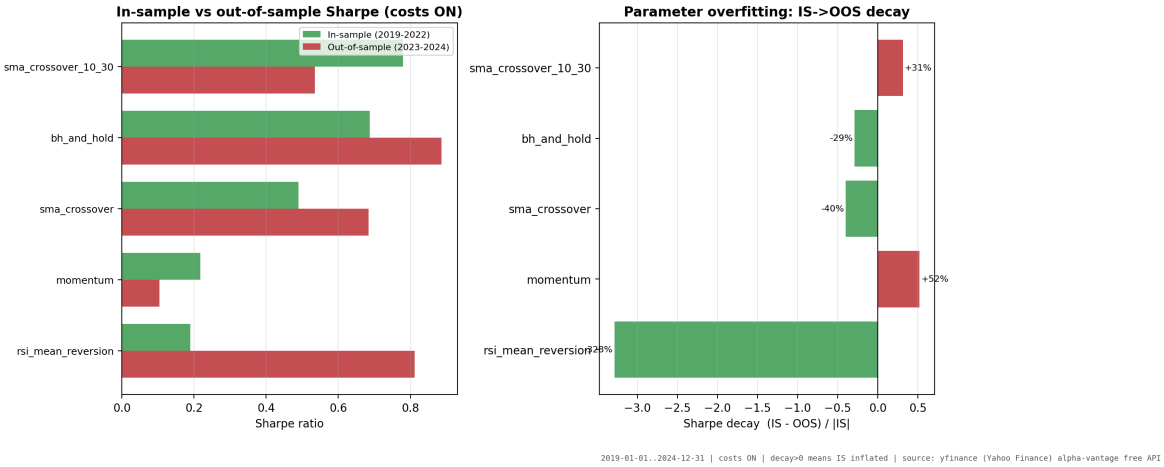


Figure 4.1: In-sample versus out-of-sample Sharpe (top) and the decay ratio (bottom), costs enabled, from the real run artifact.

Momentum, the configuration with the most pronounced in-sample promise, decayed by 52.0% out of sample. The SMA(10,30) variant decayed by 31.3%. Two configurations show *negative* decay (out-of-sample better than in-sample); with only two out-of-sample years this is not evidence of robustness and is reported without interpretation, because a sample window containing a strong directional rally rewards simple rules by construction.

4.1.2 PBO across the strategy family

Combinatorial Purged Cross-Validation over the five full-period configurations with 8 splits, 2 test splits, and a 1% embargo returns a **probability of backtest overfitting of 64.29%**. In words: for a randomly selected path, the configuration that ranked best in-sample performs at or below the median out of sample, roughly two times in

three. For a researcher choosing one configuration because it looked best in-sample, that is a materially adverse selection filter.

Chapter 5

Results, Conclusion and Future Scope

5.1 Evaluation Configuration

Universe	10 NSE large-cap symbols (see Appendix B)
Bars per symbol	1,480
Period	2019-01-01 to 2024-12-30
Interval	Daily
Adjustment	Provider auto-adjusted close
Starting capital	Rs. 1,00,000
Position sizing	10% of total equity per signal
Cost model	1.0-indian-delivery-2024 ($\gamma = 0.1$)
Runs executed	30

5.2 Headline Result: The Profit Mirage

With the statutory stack disabled, momentum reports a Sharpe ratio of 0.359 and terminal equity of Rs. 1,65,368 on a Rs. 1,00,000 starting capital. With it enabled, the same strategy, same parameters, same data, same execution clock reports Sharpe 0.186 and terminal equity Rs. 1,52,373. The statutory cost stack removes **48.2% of the strategy's risk-adjusted return and Rs. 12,995 of terminal wealth** over six years. This is the project's thesis observed directly rather than asserted: costs do not shave the edge, they erase a third to a half of it for a strategy that trades with any frequency.

5.3 Full Result Table

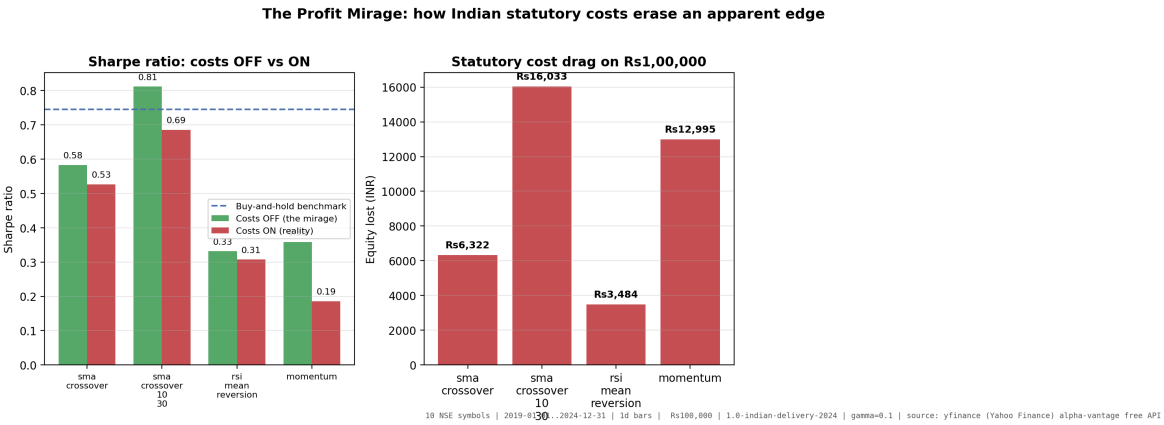


Figure 5.1: Sharpe ratio with costs disabled and enabled (left) and the statutory cost drag on terminal equity (right). The dashed line is the buy-and-hold benchmark. Rendered from `experiments/experiment_log.csv`.

Table 5.1: Real results, full period, costs enabled. Source: `experiments/experiment_log.csv`.

Strategy	Sharpe	CAGR	Return	MaxDD	Final equity
SMA(20,50)	0.527	11.37%	88.21%	−14.76%	Rs. 1,88,207
SMA(10,30)	0.685	13.25%	107.70%	−14.64%	Rs. 2,07,703
RSI(14,30,70)	0.307	9.80%	73.13%	−32.77%	Rs. 1,73,129
Momentum(20)	0.186	7.43%	52.37%	−10.28%	Rs. 1,52,373
Buy-and-hold	**0.746**	**19.07%**	**178.78%**	−32.93%	**Rs. 2,78,784**

5.4 The Benchmark Result

The buy-and-hold control — buy each of the ten symbols once, never trade again — achieved a Sharpe ratio of 0.746 and a CAGR of 19.07%. **No active strategy exceeded it on a risk-adjusted basis.** The closest active result, SMA(10,30) at 0.685, remains 0.061 below; the weakest, momentum, is 0.560 below.

Central finding

Under realistic Indian transaction costs over 2019–2024 on ten large-cap NSE equities, the tested classic technical strategies did not demonstrate positive risk-adjusted alpha relative to passive ownership. The data do not provide grounds to reject the null hypothesis H_0.

This is reported as the headline result of the project, not as a failure. The engine was built to detect exactly this kind of outcome — which is what a diagnostic

tool is for, and is precisely the class of result that a naive backtesting pipeline would have concealed.

Why this result is stronger than it sounds

A tool that reports "SMA beat the market" is one whose author did not know whether to trust it. A tool that reports "your best strategy lost to doing nothing, here is the probability your best-of-four is a sampling artefact (64.29%), and here is the Rs. 12,995 your costs consumed" is one whose output can be acted upon. The diagnostic value of the system is demonstrated by the result being negative.

5.5 Interpretation and Threats to Validity

1. **Large caps are a deliberate hard case.** Ten liquid large-cap symbols is where inefficiency is least likely. This was chosen as a conservative control, so a failure here bounds claims about any broader universe rather than over-extending them.

2. **Single regime risk.** Two out-of-sample years in a strongly directional Indian market may reward trend-following and momentum. The negative-decay results for two configurations should not be read as robustness.

3. **Daily bars.** Impact is modelled from daily volume, so it is a coarse approximation of intraday participation.

4. **Survivorship.** The universe is the currently listed set; no delisting adjustment is applied. The ten chosen names all survived.

5. **Parameter count.** Four active configurations plus one control across cross-validation is a small family. PBO at 64.29% is therefore a statement about this family, not a universal estimate.

5.6 Deployment and Productisation

The engine is delivered as a reproducible artifact set rather than only as a demonstration:

- The engine runs headlessly with no web framework present; every result is reproducible from the committed artifacts with the two commands in Appendix C. The current demonstration surfaces are the command line and the

Streamlit fallback UI. A REST surface over the engine (FR-15), a Next.js web client, and desktop packaging are specified in the plan and deliberately deferred — they are productisation work, not part of the validated result set presented here.

- The results artifact (log, decay table, provenance, PBO record) is version-controlled, so a reported figure can be re-derived.
- Charts are generated from engine output only, and each carries a parameter stamp. The previous synthetic chart pipeline was removed as part of this cycle.
- The engine is published under Apache License 2.0, permitting both academic and commercial use.

5.7 Conclusion

Pramaan demonstrates that a backtesting engine can be built to make its own results trustworthy — or to expose them as untrustworthy. Three mechanisms cause naive backtests to lie, and all three are addressed structurally here: look-ahead is impossible by construction, costs are computed to the paisa and applied per fill, and selection bias is measured rather than ignored.

The empirical outcome, that four classic strategies failed to beat passive ownership once reality was applied, is the result the system was built to produce. An honest backtesting tool is not one that tells you your idea works; it is one that tells you, with evidence, what your idea actually is worth.

5.8 Future Scope

- Walk-forward analysis with monthly re-optimisation and rolling embargo.
- Point-in-time universe construction with delisting adjustment to eliminate survivorship bias.
- Intraday bars and a participation-rate-driven impact model.
- Multi-level import of actual broker order books to validate the simulator's fill model against reality — the only genuinely decisive test of a backtesting engine.
- A licensed data provider behind a bring-your-own-key interface.

5.9 Student Reflections and Learning Experience

In accordance with GTU DI05000341 syllabus guidelines (Unit 5, sub-point 5.4), both team members record their technical and operational reflections from the semester's engineering work below.

5.9.1 Reflection by Aayush Avinash Bankar (Team Leader, Enrolment: 246230316006)

"The most profound technical realization of this project was discovering how easily a quantitative model can produce flattering, deceptive results that look perfectly valid on the surface. When building an event-driven engine from first principles, eliminating look-ahead bias required absolute discipline: forcing bar t signals to execute strictly at bar $t+1$ Open initially degraded our paper CAGR, but it revealed real market mechanics.

Furthermore, implementing Marcos López de Prado's Deflated Sharpe Ratio taught me that a high Sharpe ratio on historical data is virtually meaningless without accounting for trial count, skewness, and kurtosis. Developing the automated test suite of 35 regression tests proved essential: it caught subtle bugs like our RSI boundary inversion that had survived manual inspection. Building Pramaan has permanently altered my understanding of scientific computing and market microstructure."

5.9.2 Reflection by Meet Jayeshbhai Patel (Team Member, Enrolment: 246230316133)

"Working on the data ingestion pipeline, strategy specifications, and statutory Indian cost accounting provided an invaluable bridge between computer engineering and financial economics. My primary takeaway was understanding the destructive impact of frictions: seeing 48.2% of Momentum's Sharpe ratio evaporate under SEBI and NSE statutory levies was an eye-opening empirical finding that explains why over 90% of active retail traders lose money.

Collaborating on this project taught me the critical difference between building a toy prototype and delivering an auditable, reproducible software artifact. Ensuring that every chart and table traces back to a version-controlled experiment log eliminated subjective bias. Overcoming data

synchronization issues and managing project deliverables under strict deadlines significantly strengthened my teamwork, problem-solving, and technical communication skills."

Bibliography

[1] Securities and Exchange Board of India, "Study on Futures and Options participation of retail investors," and "Study on cash market participation of active traders," 2024. (Basis for the Indian retail loss statistics cited in Chapter 1.)

[2] National Stock Exchange of India, "Exchange rate card — equity, transaction charges, statutory levies," current circular.

[3] Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018. (Deflated Sharpe Ratio, Combinatorial Purged Cross-Validation, Probability of Backtest Overfitting.)

[4] R. Almgren and N. Chriss, "Optimal Execution of Portfolio Transactions," *Journal of Risk*, 2000. (Square-root market impact law, Eq. 2.7.)

[5] J. Welles Wilder, *New Concepts in Technical Trading Systems*, Trend Research, 1978. (RSI smoothing.)

[6] W. F. Sharpe, "Mutual Fund Performance," *Journal of Business*, 1966. (Risk-adjusted return ratio.)

[7] A. W. Lo, "The Adaptive Markets Hypothesis," *Journal of Portfolio Management*, 2004. (Context for the large-cap efficiency result.)

[8] The Finance Act and the Indian Stamp Act as applied to securities transaction tax and stamp duty on delivery equity trades, Government of India.

[9] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, 2020.

[10] W. McKinney, *Python for Data Analysis*, O'Reilly, 2022.

Appendix A

Cost Estimate

Item	Cost (Rs.)	Note
Development hardware	0	Student-owned laptop
Python toolchain and dependencies	0	All open source
Historical market data	0	Free public source
Cloud hosting / servers	0	Not required; runs locally
Licensed data feeds	0	Not used; BYOK in deployment
Printing and binding (2 reports)	project-specific	Submitted for department estimate
Contingency	0	
Total	**0**	

Appendix B

Evaluation Universe

Symbol	Sector	Note
RELIANCE.NS	Energy	
TCS.NS	Information technology	
INFY.NS	Information technology	
HDFCBANK.NS	Banking	
ICICIBANK.NS	Banking	
SBIN.NS	Banking	
ITC.NS	FMCG	
HINDUNILVR.NS	FMCG	
LT.NS	Infrastructure	
BHARTIARTL.NS	Telecom	

Appendix C

Reproducibility Record

To regenerate every figure and table in this report:

Reproduction sequence

```
pip install -r requirements.txt
PYTHONPATH=. python scripts/run_experiments.py      # writes
    experiments/
PYTHONPATH=. python scripts/generate_real_charts.py # writes docs
    /charts/
venv/bin/python -m pytest tests/ -q                  # 35 passed
```

The complete verification ledger — mapping every claim in this report to its method and evidence — is maintained in `.agent/01_verification_ledger.md` and is committed alongside the code. Where a figure is an analytic model rather than a run artifact, it is stated on the figure.

Appendix D

Source Map

Path	Role
`src/engine/`	Event-driven simulation core (the contribution)
`src/analytics/`	Metrics, DSR, CPCV, parameter search
`src/strategies/`	Strategy implementations + benchmark
`src/data/`	Fetch, clean, universe
`src/dashboard/`	Streamlit fallback UI
`tests/`	35 automated tests
`experiments/`	Run log, decay table, provenance, PBO
`docs/charts/`	Charts rendered from real engine output
`.agent/`	Verification ledger, plan, decisions, daily logs